

CPU Scheduling Algorithm

소속 영남대학교 IT학부 컴퓨터공학과

학번 22212053 22213487 22213501

이름 김수정 권기범 김주현

[요 약]

본 프로젝트는 CPU 스케줄링 알고리즘의 이론적 이해를 바탕으로 실제로 구현하고, 각 알고리즘의 성능을 비교 분석하여 운영체제의 핵심 기능을 심층적으로 이해하는 것을 목표로 한다.

본 프로젝트를 위해서는 운영체제 기본 개념과 다양한 스케줄링 알고리즘의 이해가 필요하며, 명시한 알고리즘의 기능 및 동작을 바탕으로 실험 환경을 통해 실험 결과를 도출한다. 이를 기반으로 본 프로젝트의 핵심 결론과 향후 알고리즘 개선 방안을 제시한다.

I. 서론

본 프로젝트는 운영체제 학습의 일환으로, 프로세스 스케줄러를 설계하고 구현하는 것을 목표로 한다. 프로세스 스케줄러는 운영체제의 핵심 구성 요소 중 하나로, 여러 프로세스가 CPU를 효율적으로 사용할 수 있도록 관리하는 역할을 한다. 스케줄러는 다양한 알고리즘을 통해 프로세스의 우선순위를 결정하고, 이를 기반으로 CPU 시간을 적절히 분배함으로써 시스템의 전반적인 성능과 응답성을 향상시킨다. 이번 프로젝트에서는 FCFS, Shortest-Job-First Scheduling, Priority Scheduling, Round-Robin Scheduling과 같은 대표적인 스케줄링 알고리즘을 구현하고, 각 알고리즘의 성능을 비교 분석할 것이다.

프로세스 스케줄러 설계는 운영체제의 핵심 기능을 깊이 이해하고, 실제 시스템에서의 적용 방법을 배우는 중요한 학습 과정이다. 이를 통해 학생들은 운영체제의 기본 원리 이해, 알고리즘 분석 및 구현 능력 향상과 같은 중요한 개념과 기술을 습득할 수 있다. 따라서 본 프로젝트는 단순한 이론 학습을 넘어, 실제 운영체제의 중요한 기능을 직접 구현하고 분석하는 경험을 제공함으로써, 학생들의 전공 역량을 한층 더 강화하는 데 큰 의의가 있다.

본 프로젝트의 주요 목표는 프로세스 스케줄링 알고리즘을 이해 및 구현하고 스케줄링 알고리즘의 성능 비교 및 분석함으로써, 각 스케줄링 알고리즘이 어떻게 CPU 자원을 효율적으로 관리하는지 확인하는 것이다.

본 프로젝트에서는 운영체제의 프로세스 스케줄러를 설계하고 구현함으로써, 프로세스 생성 및 관리, 다양한 스케줄링 알고리즘 구현, 프로세스 우선순위 관리, CPU 스케줄링 시뮬레이션, 성능 측정 및 분석과 같은 핵심 기능을 제공한다. 이러한 기능들을 통해, 사용자는 운영체제의 프로세스 스케줄링 메커니즘을 깊이 이해하고, 각 알고리즘의 장단점을 실제 시뮬레이션을 통해 확인할 수 있다.

II. 배경 지식 및 관련 기술

1. 배경 지식

본 프로젝트를 성공적으로 수행하기 위해서는 운영체제 기본 개념, 프로세스 스케줄링의 목적과 중요성, FCFS, Shortest-Job-First Scheduling, Priority Scheduling, Round-Robin Scheduling과 같은 대표적인 스케줄링 알고리즘의 기본 원리와 작동 방식, 각 스케줄링 알고리즘의 장단점 및 적합한 사용 사례, 프로그래밍 언어 및 개발 환경, 성능 측정을 위한 다양한 지표(CPU 수행 시간, 도착 시간, 대기 시간, 응답 시간 등)의 의미와 계산 방법, 구현한 스케줄링 알고리즘의 성능을 평가하고 비교하는 방법, 성능 측정 결과를 그래프나 표 형태로 시각화하는 기술과 같은 배경 지식이 필요하다.

마찬가지로 본 프로젝트 문서를 참고하는 사람 또한 이해를 돕기 위해 기본적으로 알고 있어야 하는 배경 지식으로 운영체제 기본 개념, FCFS, Shortest-Job-First Scheduling, Priority Scheduling, Round-Robin Scheduling과 같은 대표적인 스케줄링 알고리즘의 기본 원리와 작동 방식, 프로그래밍 언어 및 개발 환경, 성능 측정을 위한 다양한 지표(CPU 수행 시간, 도착 시간, 대기 시간, 응답 시간 등)의 의미와 계산 방법 등이 있다. 이러한 배경 지식을 바탕으로 문서를 참고하는 사람은 프로젝트의 내용을 보다 쉽게 이해할 수 있으며, 구현된 프로세스 스케줄러의 작동 원리와 성능 분석 결과를 효과적으로 파악할 수 있다.

2. 관련 기술

2.1 First-Come-First-Serve (FCFS) Scheduling

FCFS 스케줄링은 가장 간단한 스케줄링 알고리즘으로, 도착 순서대로 프로세스를 처리한다. 공정성과 구현의 단순성 덕분에 교육용 및 단순 시스템에서 여전히 사용되고 있다. 그러나 긴 프로세스가 앞에 있을 경우 발생하는 Convoy Effect로 인해, 실제 고성능 시스템에서는 거의 사용되지 않는다.

2.2 Shortest-Job-First (SJF) Scheduling

SJF는 평균 대기시간을 최소화하는 이론적으로 최적의 알고리즘이다. 그러나 각 프로세스의 실행 시간을 정확히 예측하기 어려워 실제 구현에서는 변형된 알고리즘이 사용된다. 또한, 짧은 작업이 계속해서 도착할 경우 긴 작업이 무기한 대기하게 되는 Starvation 문제가 발생할 수 있다.

2.3 Priority Scheduling

Priority Scheduling은 중요한 작업을 우선 처리하는 방식으로, 실시간 시스템에서 널리 사용된다. 프로세스에 우선 순위를 부여하고, 우선순위가 높은 프로세스를 먼저 처리한다. 그러나 우선순위가 낮은 프로세스가 무기한 대기할 수 있는 Starvation 문제를 해결하기 위해 Aging 기법이 도입되었다.

2.4 Round-Robin (RR) Scheduling

RR 스케줄링은 각 프로세스에 동일한 시간 할당량을 주어 순환하면서 처리한다. 공정성과 빠른 응답 시간을 보장하기 때문에 사용자 인터랙션이 중요한 멀티태스킹 시스템에서 널리 사용된다. 타임 퀀텀의 크기가 성능에 큰 영향을 미치며, 이를 적절히 설정하는 것이 중요하다.

2.5 배치 처리 시스템 (Batch Processing Systems)

배치 처리 시스템은 초기 컴퓨터 시스템에서 사용된 기술로, 여러 작업을 모아서 한꺼번에 처리하는 방식이다. 사용자가 작업을 제출하면, 시스템은 이 작업들을 순차적으로 처리한다. 이러한 시스템에서는 주로 First-Come-First-Serve(FCFS) 스케줄링 알고리즘을 사용한다. 배치 처리 시스템은 작업이 완료될 때까지 사용자와의 상호작용이 없으며, 오직 결과만을 제공받는다. 이 방식은 효율적인 자원 사용을 목표로 하나, 긴 대기 시간과 사용자 응답 시간 문제로 인해 제한적인 사용이 된다. 배치 처리 시스템은 초기 컴퓨팅 환경에서 주로 사용되었으며, 현대의 운영체제 스케줄링 알고리즘 개발에 기초가 되었다.

2.6 시분할 시스템 (Time-Sharing Systems)

시분할 시스템은 여러 사용자가 동시에 시스템을 사용할 수 있도록 하는 것을 위해 개발된 기술이다. 시스템은 각 사용자에게 일정 시간 동안 CPU를 할당하고, 이를 반복하여 사용자들이 동시에 시스템을 사용하는 것처럼 보이게 한다. 시분할 시스템은 Round-Robin(RR) 스케줄링 알고리즘을 기반으로 하며, 사용자가 빠른 응답 시간을 경험할 수 있도록 설계되었다. 이 시스템은 CPU 시간을 공정하게 분배하여 모든 사용자에게 균등한 처리 기회를 제공한다. 시분할 시스템은 현대의 멀티태스킹 운영체제의 기초가 되며, 사용자 인터랙션이 중요한 시스템에서 널리 사용된다.

2.7 멀티프로그래밍 시스템 (Multiprogramming Systems)

멀티프로그래밍 시스템은 여러 프로그램이 동시에 메모리에 적재되어 실행되는 방식이다. 이는 CPU 사용률을 극대화하기 위해 개발되었다. 멀티프로그래밍 시스템에서는 CPU가 한 프로그램의 I/O 작업을 기다리는 동안 다른 프로그램을 실행함으로써 자원을 효율적으로 사용한다. 스케줄링 알고리즘으로 FCFS, SJF, Priority Scheduling 등을 사용할 수 있다. 멀티프로그래밍 시스템은 CPU 자원의 효율적 사용을 통해 시스템 성능을 향상시키는 방법을 제시했으며, 이는 현대 운영체제의 멀티태스킹 기능의 기반이 되었다.

2.8 실시간 시스템 (Real-Time Systems)

실시간 시스템은 특정 시간 내에 작업을 완료해야 하는 요구사항을 가진 시스템이다. 이러한 시스템은 주로 산업 제어, 의료 기기, 통신 장비 등에서 사용된다. 실시간 시스템에서는 정해진 시간 내에 작업을 완료하기 위해 EDF(Earliest Deadline First), RMS(Rate-Monotonic Scheduling)와 같은 실시간 스케줄링 알고리즘이 많이 사용된다. 실시간 시스템은 엄격한 시간 제약을 만족시키기 위해 스케줄러가 프로세스의 우선순위를 동적으로 조정해야 한다. 실시간 시스템은 시간 제약이 중요한 환경에서의 운영체제 설계에 큰 영향을 미쳤으며, 이는 실시간 요구사항을 만족시키는 다양한 스케줄링 알고리즘 개발로 이어졌다.

2.9 유닉스 스케줄러 (Unix Scheduler)

유닉스 운영체제는 초기부터 다양한 스케줄링 알고리즘을 사용하여 프로세스 관리 기능을 제공했다. 유닉스 스케줄러는 주로 멀티태스킹 환경에서의 성능 최적화를 목표로 한다. 유닉스 스케줄러는 기본적으로 우선순위 기반 스케줄링을 사용하며, 프로세스의 CPU 사용 패턴에 따라 동적으로 우선순위를 조정한다. 또한, 인터랙티브 작업과 배치 작업을 효율적으로 처리하기 위해 다양한 기법을 적용한다. 유닉스 스케줄러는 현대 운영체제의 다양한 스케줄링 알고리즘의 기초가 되었으며, 리눅스와 같은 많은 운영체제에서 유사한 방식이 사용되고 있다.

III. CPU Scheduling Algorithm

1. 개요

본론에서는 운영체제의 CPU 스케줄링 알고리즘에 대해 설명하고, 고급 프로그래밍 언어로 구현하고자 한다. 컴퓨팅에서 스케줄링이란 다중 프로그래밍을 가능하게 하는 운영체제의 동작 기법이다. 운영체제는 프로세스에게 CPU와 같은 자원을 적절히 배정하여 응답 시간과 처리량의 최적화와 같이 시스템의 성능을 개선하는 것이 스케줄링의 역할이다.

CPU 스케줄링의 주요 목적은 가능한 한 많은 작업(또는 프로세스)을 동시에 실행하도록 하는 것이다. 하지만 CPU 자원은 한정적이기 때문에 다수의 작업이 동시에 실행되는 것은 어렵다. 한정적인 자원으로 CPU의 효율적인 사용과 모든 작업에 CPU 시간을 공정하게 배분하여 실행할 수 있어야하므로 CPU 스케줄링은 필수이다.

위에서 언급하였듯, CPU 스케줄링의 목표는 가능한 한 많은 작업을 동시에 실행하도록 하는 것이다. 즉, 모든 프로세스에 CPU 시간을 공정하게 배분하여, 단위 시간당 최대한 많은 프로세스를 완료하는 것이다. 또한, 사용자 요청에 신속하고 응답하고, 프로세스가 대기열에서 대기하는 시간을 최소화하는 것이다.

CPU 스케줄링의 유형은 대표적으로 선점형과 비선점형으로 나뉘고, 선점형 스케줄링은 실행 중인 프로세스를 중단하고 다른 프로세스를 실행하는 방식이고, 비선점형 스케줄링은 현재 실행 중인 프로세스가 완료될 때까지 다른 프로세스로 전환하지 않는 방식이다.

본 프로젝트에서 다루는 CPU 스케줄링 알고리즘은 널리 알려진 네 가지 CPU 스케줄링 알고리즘인 First-Come, First-Served (FCFS), Shortest Job First (SJF), Priority Scheduling, 그리고 Round Robin (RR)에 대해 다룰 것이고, 각 스케줄링 알고리즘의 선점형과 비선점형 형태를 알아볼 것이다. 각 알고리즘의 개념과 작동 원리에 대해 살펴보고 이해하고자 한다.

2. CPU Scheduling Algorithm의 종류 및 기능

2.1 First-Come, First-Served (FCFS) Scheduling

First-Come, First-Served (FCFS) 스케줄링은 가장 단순한 CPU 스케줄링 알고리즘이다. 먼저 도착한 프로세스가 먼저 처리되는 시스템이다. 단순하기 때문에 결과를 쉽게 예측할 수 있고, 구현이 간단하다. 또한 알고리즘의 진행이 선착순이기 때문에 공정하다. 하지만 Convoy effect (호위 효과)로 인해 실행 시간이 긴 프로세스가 먼저 도착하여 실행될 경우, 늦게 도착한 프로세스들의 대기 시간이 길어지게 될 수 있다. 이 문제는 시스템의 자원 활용도를 저하시키고, 사용자 요청에 대한 응답 시간을 지연시킨다.

각 프로세스는 도착한 순서대로 대기열(Ready Queue)에 정렬하고, CPU가 비어있다면 대기열의 가장 앞에서 대기 중인 프로세스가 CPU 자원을 할당받아 실행된다. FCFS 스케줄링은 비선점형 스케줄링이기 때문에, 실행 중인 프로세스는 실행이 종료될 때까지 중단되지 않고, 프로세스가 완전히 처리된 후 다음 프로세스가 실행되는 식으로 동작한다.

FCFS 의사 코드

```
function FCFS(processes):
    # processes는 (PID, AT, BT, Priority)로 구성된 구조체 배열
    processes를 AT가 작은 순서로 정렬

    currentTime, doneTask = 0
    numProcess = 프로세스의 수
    readyQueue # 대기열을 구현하기 위한 구조체 배열, 선입선출의 큐 형식이다.
    runProcess # 실행 중인 프로세스를 구현하기 위한 구조체 배열

    jobResult # (PID, BT, WT)로 구성된 구조체 배열

    while doneTask < numProcess:
        for process in Processes:
            if process.AT == currentTime:
                readyQueue = process

            if runProcess.PID == -1;
                runProcess = (readyQueue)를 매개 변수로 AT가 가장 작은 프로세스를 반환하는 함수

            if runProcess.BT != 0:
                runProcess.Bt -=1
                if runProcess.BT == 0:
                    doneTask += 1
                    runProcess 초기화

        currentTime += 1
```

2.2 Shortest Job First (SJF) Scheduling

Shortest Job First (SJF) 스케줄링 알고리즘은 대기열에 있는 프로세스 중 실행 시간이 가장 짧은 프로세스를 우선으로 실행하는 스케줄링 알고리즘이다. 대기열 중 가장 짧은 프로세스를 우선으로 실행하기 때문에 평균 대기시간을 최소화하는 데 효과적이지만, 실행 시간이 긴 프로세스는 영원히 실행되지 않는 Starvation(기아) 문제가 발생할 수 있고, 프로세스 실행 시간을 미리 알아야 한다는 점 때문에 구현이 어려워 다른 스케줄링 알고리즘과의 비교군으로 사용한다는 특징이 있다. SJF 스케줄링 알고리즘은 비선점형과 선점형 두 가지 방식으로 구현될 수 있다.

2.2.1 Non-Preemptive SJF Scheduling

비선점형 SJF(이하 SJF) 스케줄링 알고리즘은 프로세스가 대기열에 도착하고, CPU가 빈 상태가 되면 대기열에서 실행 시간이 가장 짧은 프로세스가 실행된다. 실행된 프로세스는 실행을 완료할 때까지 자원을 사용한다. 프로세스가 완료 되면 대기열에서 실행 시간이 가장 짧은 다음 프로세스가 실행된다.

SJF 스케줄링 알고리즘은 다른 스케줄링 알고리즘보다 평균 대기시간이 짧다는 장점이 있지만, 비선점형 스케줄링의 특성상 실행 시간이 긴 프로세스가 먼저 도착하면 다음에 도착한 프로세스가 오래 기다려야 하는 단점이 있다.

2.2.2 Preemptive SJF (Shortest Remaining Time First, SRTF) Scheduling

SRTF 스케줄링 알고리즘은 SJF 스케줄링이 선점형 스케줄링으로 처리되는 스케줄링 알고리즘이다. 실행 중인 프로세스를 포함하여 남은 시간이 가장 짧은 프로세스를 선택하여 CPU를 할당하는 방식이다. 항상 가장 중요한 프로세스를 실행하기 때문에 평균 대기시간이 짧아지고 CPU 자원을 효율적으로 사용할 수 있지만, 새로운 프로세스가 실행되는 과정인 Context switch (문맥 교환)이 발생하게 되고, 문맥 교환의 빈도가 높아지면 오버헤드가 증가한다. 오버헤드란 문맥 교환을 하는 동안 시스템이 작업을 하지 못하는 시간을 말한다. 오버헤드가 높아지면 시스템의 성능이 저하되기 때문에 최소화가 필요하다. 또한 SJF 스케줄링 알고리즘과 마찬가지로 프로세스의 실행 시간을 미리 알아야 하는 어려움이 있다.

SRTF 의사 코드

```
function Priority(processes):
    # processes는 (PID, AT, BT, Priority)로 구성된 구조체 배열
    processes를 AT가 작은 순서로 정렬

    currentTime, doneTask = 0
    numProcess = 프로세스의 수
    readyQueue # 대기열을 구현하기 위한 구조체 배열, 선입선출의 큐 형식이다.
    runProcess # 실행 중인 프로세스를 구현하기 위한 구조체 배열

    jobResult # (PID, BT, WT)로 구성된 구조체 배열

    while doneTask < numProcess:
        for process in Processes:
            if process.AT == currentTime:
                readyQueue = process

        if runProcess.PID == -1;
            runProcess = (readyQueue)를 매개 변수로 BT가 가장 작은 프로세스를 반환하는 함수
        else:
            for process in readyQueue:
                if runProcess.BT > process.BT:
                    temp = runProcess
                    runProcess = process
                    readyQueue = temp

            if runProcess.BT != 0:
                runProcess.BT -= 1
            if runProcess.BT == 0:
                doneTask += 1
                runProcess 초기화

    currentTime += 1
```


2.3 Priority Scheduling

Priority 스케줄링 알고리즘은 각 프로세스에 우선순위를 부여하고, 우선순위가 높은 프로세스를 먼저 실행하는 스케줄링 알고리즘이다. 앞서 소개되었던 SJF 스케줄링 알고리즘도 실행 시간에 따라 먼저 처리될 프로세스가 정해지므로, Priority 스케줄링 알고리즘의 일종으로 볼 수 있다.

Priority 스케줄링 알고리즘의 문제점은 우선순위가 낮은 프로세스는 실행 순서가 계속해서 뒤로 밀려, Starvation (기아) 문제가 발생할 수 있다. 이 문제는 시간이 진행됨에 따라, 시스템이 프로세스의 우선순위를 높이는 Aging(노화) 기법으로 해결할 수 있고, 이 기법을 SJF 스케줄링 알고리즘에 추가시킨 High Response Ratio Next (HRRN) Scheduling 이 있지만, 이 보고서에는 다루지 않는다.

2.3.1 Non-Preemptive Priority Scheduling

Non-preemptive Priority 스케줄링 알고리즘은 비선점형 스케줄링이기 때문에 실행 중인 프로세스는 중단할 수 없고, 새로운 프로세스가 이미 실행 중인 프로세스보다 우선순위가 높지만, 자원을 선점할 수 없기 때문에 Priority Inversion(우선순위 역전 현상)이 발생할 수 있다.

2.3.2 Preemptive Priority Scheduling

Preemptive Priority 스케줄링 알고리즘은 비선점형 우선순위 스케줄링 알고리즘과의 차이는 새로운 프로세스가 대기열에 도착할 때 마다 실행 중인 프로세스와의 우선순위를 비교하여 새로운 프로세스의 우선순위가 높은 경우 실행 중인 작업을 중단하고 새로운 프로세스를 실행한다. 선점형 우선순위 스케줄링 알고리즘은 새로운 프로세스가 실행되는 과정에서 문맥 교환이 발생할 수 있다. 또한, 비선점형 우선순위 스케줄링 알고리즘과 마찬가지로 프로세스의 우선순위가 낮은 경우 프로세스의 실행 순서가 뒤로 밀리게 되고, 기아(Starvation) 문제가 발생할 수 있다.

Priority 의사 코드

```
function Priority(processes):
    # processes는 (PID, AT, BT, Priority)로 구성된 구조체 배열
    # processes를 AT가 작은 순서로 정렬

    currentTime, doneTask = 0
    numProcess = 프로세스의 수
    readyQueue # 대기열을 구현하기 위한 구조체 배열, 선입선출의 큐 형식이다.
    runProcess # 실행 중인 프로세스를 구현하기 위한 구조체 배열

    jobResult # (PID, BT, WT)로 구성된 구조체 배열

    while doneTask < numProcess:
        for process in Processes:
            if process.AT == currentTime:
                readyQueue = process

        if runProcess.PID == -1;
            runProcess = (readyQueue)를 매개 변수로 Priority가 가장 작은 프로세스를 반환함
        else:
            for process in readyQueue:
                if runProcess.Priority > process.Priority:
                    temp = runProcess
                    runProcess = process
                    readyQueue = temp

            if runProcess.BT != 0:
                runProcess.BT -= 1
            if runProcess.BT == 0:
                doneTask += 1
                runProcess 초기화

    currentTime += 1
```

2.4 Round Robin (RR) Scheduling

Round Robin 스케줄링 알고리즘은 First-In, First-Out (FIFO) 알고리즘을 기반으로 한 스케줄링 알고리즘이며, 선점형 FCFS 스케줄링 알고리즘이라고 볼 수 있다. 프로세스에게 한 번에 실행할 수 있는 할당량인 타임 쿼텀(Time Quantum)을 제공하고, 타임 쿼텀을 기준으로 프로세스가 순환하며 처리되는 스케줄링 알고리즘이다. RR 스케줄링 알고리즘에서는 프로세스의 도착 시간이 모두 같다고 가정하고, 대기열은 원형 큐의 형태로 구현된다. 각 프로세스는 타임 쿼텀만큼 실행되고, 할당된 시간이 끝나면 대기열의 끝으로 이동한다.

RR 스케줄링 알고리즘은 프로세스 실행 시간에 공정성이 보장되어 모든 프로세스가 할당된 시간 내로 자원을 사용할 수 있어, 빠른 응답 시간을 필요로 하는 시분할 시스템에 사용된다. RR 스케줄링 알고리즘은 프로세스가 일정한 시간 내에 CPU를 할당받을 수 있게 하여 사용자 명령에 빠른 응답을 보장한다. 하지만 타임 쿼텀이 너무 커지면 오히려 FCFS 스케줄링 정책의 하위 호환에 불과하고, 타임 쿼텀이 너무 작아지면 문맥 교환이 잦아져 오버헤드가 커지는 것을 조심해야 한다.

RR 의사 코드

```
function Priority(processes):
    # processes는 (PID, AT, BT, TimeQuantum)로 구성된 구조체 배열
    # processes를 AT가 작은 순서로 정렬

    currentTime = 0, doneTask = 0
    numProcess = 프로세스의 수
    readyQueue # 대기열을 구현하기 위한 구조체 배열, 선입선출의 큐 형식이다.
    runProcess # 실행 중인 프로세스를 구현하기 위한 구조체 배열

    jobResult # (PID, BT, WT)로 구성된 구조체 배열

    while doneTask < numProcess:
        for process in Processes:
            if process.AT == currentTime:
                readyQueue = process

        if runProcess.PID == -1:
            runProcess = (readyQueue)를 매개 변수로 배열의 첫 번째 값을 반환하는 함수
        else if runProcess.PID != -1 && TimeQuantum == 0:
            for process in readyQueue:
                if runProcess.BT > process.BT:
                    temp = runProcess
                    runProcess = process
                    readyQueue = temp

        if runProcess.BT != 0:
            runProcess.BT -= 1
            runProcess.TimeQuantum -= 1
        if runProcess.BT == 0:
            doneTask += 1
            runProcess 초기화

    currentTime += 1
```

3. 구현과 동작 사례

이 보고서에서는 기본적인 스케줄링 알고리즘인 FCFS, SRTF, Preemptive Priority, 그리고 RR을 구현하고 동작 사례에 대해 알아볼 것이다. 본문에서는 프로세스의 평균 대기시간을 기준으로 스케줄링 정책을 비교하였다.

구현하기에 앞서, 동작 사례를 이해하기 위한 수의 단위가 크지않은 간단한 입력값을 준비하였다. 아래의 예시에서는 프로세스의 실행 시간이 크지 않고, 편차도 적지 때문에 FCFS 스케줄링 알고리즘의 평균 대기시간은 크지 않을 것으로 예상할 수 있고, Priority 스케줄링 알고리즘과 RR 스케줄링 알고리즘에서는 문맥 교환 오버헤드로 인해 성능이 저하될 것으로 예상할 수 있다. 하지만 제일 처음 대기열에 도착하는 프로세스의 실행 시간이 극단적으로 커지는 경우에는 결과는 충분히 달라질 수 있다.

Process ID	Arrival Time	Burst Time	Priority
1	1	7	2
2	0	10	7
3	4	6	5
4	7	4	2
5	6	5	1

아래의 코드는 모든 소스 코드에 공통적으로 포함되는 부분이다.

```
// 프로세스 구조체
typedef struct {
    int processID;
    int arrivalTime;
    int burstTime;
    int priority;
}Process;

// 결과 출력 구조체
typedef struct {
    int processID;
    int burstTime;
    int waitingTime;
} ResultElement;
```

프로세스를 구현하기 위한 구조체와 실행 시간과 대기 시간 등 결과를 출력하기 위한 구조체이다.

```
// 대기열 생성
readyQueue = (Process *)malloc(sizeof(Process) * (numProcess+5));
if (readyQueue == NULL) {
    fprintf(stderr, "메모리 할당을 실패했습니다.");
    exit(-1);
}
// 대기열 초기화
for (int i = 0; i < numProcess+5; i++)
    Initialize(&readyQueue, i);

runProcess = (Process*)malloc(sizeof(Process));
Initialize(&runProcess, 0); // 실행 중인 프로세스가 없다는 것을 의미
```

위 코드는 프로세스가 입장할 대기열과 프로세스가 실행되는 것을 구현하기 위한 코드이다. 구조체 Process를 포인터로 생성한 구조체 변수 readyQueue를 동적할당 하여 대기열을 구현하였고, 대기열에 입장할 프로세스를 위해 대기열을 초기화하였다. 실행될 프로세스를 다음 구조체 변수 runProcess 또한 초기화를 진행하였다.

```

while (doneProcess < numProcess) {
    // 프로세스가 대기열로 진입
    for (int i = 0; i < numProcess; i++) {
        if (readyQueue[queueIndex].processID == -1 && process[i].processID != -1 && currentTime == process[i].arrivalTime) {
            readyQueue[queueIndex] = process[i];
            queueIndex++;
            newProcess++;
        }
    }
}

```

스케줄링 알고리즘은 종료된 프로세스의 수가 모든 프로세스의 수보다 많으면 종료된다. 위 코드의 조건문은 프로세스가 대기열로 진입하기 위한 코드이다. 현재 CPU 시간과 프로세스의 도착 시간이 같으면 프로세스는 대기열로 입장한다.

```

// 프로세스 실행
if (runProcess->burstTime != 0) {
    for (int i = 0; i < numProcess; i++) {
        if (runProcess->processID == process[i].processID) {
            showProcess[i].processID = runProcess->processID;
            showProcess[i].burstTime = process[i].burstTime - (--runProcess->burstTime);
            total.burstTime++;
            // 실행 프로세스 종료
            if (runProcess->burstTime == 0) {
                for (int j = 0; j < numProcess; j++) {
                    if (showProcess[j].processID == runProcess->processID) {
                        ShowResult(showProcess[j]);
                        gantt[ganttIndex] = showProcess[j];
                        ganttIndex++;
                        gantt = realloc(gantt, sizeof(ResultElement) * (ganttIndex+1));
                    }
                }
                doneProcess++;
                Initialize(&runProcess, 0);
            }
        }
    }
}
}

```

프로세스를 실행하고, 실행 후 종료됐다면 종료된 프로세스의 수를 1 씩 더하는 코드이다. 프로세스가 실행되면 실행 중인 프로세스의 실행 시간은 1 씩 감소하고, 결과를 보여주기 위한 구조체 변수 showProcess와 실행 중인 프로세스의 프로세스 ID가 같으면 showProcess의 실행 시간과 대기 시간을 1 씩 추가하여 총 실행 시간과 평균 대기 시간을 출력할 수 있도록 코드를 작성하였다. 프로세스의 실행 시간이 0이 되어 프로세스가 종료된 경우에는 다음에 실행될 프로세스를 위해 자원을 비우는 코드를 작성하였다.

자원을 할당받은 프로세스가 실행될 동안, 대기열에서 대기 중인 프로세스의 대기 시간을 추가하는 코드이다. while 반복문의 한 사이클이 지나면 CPU 시간을 추가하는 코드도 작성하였다.

3.1 FCFS Scheduling

```
// 대기 중인 프로세스 대기 시간 추가
for (int i = 0; i < queueIndex; i++) {
    for (int j = 0; j < newProcess; j++) {
        if (readyQueue[i].processID == showProcess[j].processID) {
            showProcess[j].waitingTime++;
            gantt[ganttIndex] = showProcess[j];
        }
    }
}
```

```
currentTime++;
```

먼저 가장 단순한 FCFS 스케줄링 알고리즘을 C언어로 구현한 코드이다.

```
// 실행 중인 프로세스가 없는 경우
if (runProcess->processID == -1) {
    int index = nextRun(readyQueue, queueIndex);
    *runProcess = readyQueue[index];
    Initialize(&readyQueue, index);
}
```

프로세스가 대기열로 입장한 후, 실행 중인 프로세스가 없다면 아래의 코드에서 조건에 부합하는 프로세스를 찾아 실행한다.

```
// 다음 실행될 프로세스 찾는 함수
int nextRun(Process* process, int size) {
    int index = 0;
    Process highPriority = *process;

    for (int i = 0; i < size; i++) {
        if (highPriority.processID == -1 && process[i].processID != -1) {
            highPriority = process[i];
            index = i;
            break;
        }
    }

    for (int i = 0; i < size; i++) {
        if (process[i].processID != -1) {
            if (highPriority.arrivalTime > process[i].arrivalTime) {
                highPriority = process[i];
                index = i;
            }
            else if (highPriority.arrivalTime == process[i].arrivalTime && highPriority.processID > process[i].processID) {
                highPriority = process[i];
                index = i;
            }
        }
    }

    return index;
}
```

가장 우선순위가 높은 프로세스가 다음 실행될 프로세스로 선택되는 코드이다. FCFS 스케줄링 알고리즘에서는 도착 시간이 가장 빠른 프로세스가 높은 우선순위를 가지게 된다. 오류를 방지하기 위해 도착 시간이 같을 경우에는 프로세스 ID의 숫자가 낮은 프로세스가 먼저 실행하도록 작성하였다.

앞서 언급한 스케줄링 알고리즘의 동작 사례를 위한 입력 값이다.

FCFS 스케줄링

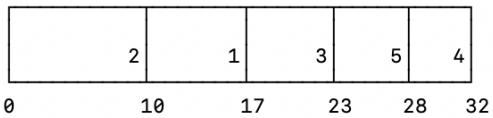
입력

Process ID	Arrival Time	Burst Time
1	1	7
2	0	10
3	4	6
4	7	4
5	6	5

스케줄링 알고리즘이 실행된 후 프로세스가 도착 시간 순서에 맞춰 실행되는 모습을 표와 간트 차트를 통해 확인할 수 있다.

FCFS 스케줄링 실행

Process ID	Burst Time	WaitingTime
2	10	0
1	7	9
3	6	13
5	5	17
4	4	21



프로세스 ID 순서로 정렬된 결과를 통해 전체 실행 시간과 평균 대기시간을 확인할 수 있다.

결과

Process ID	Burst Time	WaitingTime
1	7	9
2	10	0
3	6	13
4	4	21
5	5	17

전체 실행 시간 : 32, 평균 대기 시간 : 12.0

3.2 SRTF Scheduling

다음으로 구현할 스케줄링 알고리즘은 SRTF 스케줄링 알고리즘이다.

```
// 다음 실행될 프로세스 찾는 함수
int nextRun(Process* process, int size) {
    int index = 0;
    Process highPriority = *process;

    for (int i = 0; i < size; i++) {
        if (highPriority.processID == -1 && process[i].processID != -1) {
            highPriority = process[i];
            index = i;
            break;
        }
    }

    for (int i = 0; i < size; i++) {
        if (process[i].processID != -1) {
            if (highPriority.burstTime > process[i].burstTime) {
                highPriority = process[i];
                index = i;
            }
            else if (highPriority.burstTime == process[i].burstTime && highPriority.arrivalTime > process[i].arrivalTime) {
                highPriority = process[i];
                index = i;
            }
        }
    }

    return index;
}
```

실행 중인 프로세스가 없을 경우, 실행 시간이 적은 프로세스가 높은 우선순위를 가지고, 실행 시간이 같다면 도착 시간이 빠른 프로세스가 먼저 실행하도록 작성하였다.

```
// 실행 중인 프로세스가 없는 경우
if (runProcess->processID == -1) {
    int index = nextRun(readyQueue, queueIndex);
    *runProcess = readyQueue[index];
    Initialize(&readyQueue, index);
}
else { // 대기열에 프로세스 진입 시 실행 중인 프로세스가 있는 경우, 대기열에 진입한 프로세스의 실행 시간이 적다면 실행
    for (int i = 0; i < newProcess; i++) {
        if (readyQueue[i].processID != -1 && runProcess->burstTime > readyQueue[i].burstTime) {
            for (int j = 0; j < numProcess; j++) {
                if (runProcess->processID == showProcess[j].processID) {
                    ShowResult(showProcess[j]);
                    gantt[ganttIndex] = showProcess[j];
                    ganttIndex++;
                    gantt = realloc(gantt, sizeof(ResultElement) * (ganttIndex+1));
                }
            }
            Process temp = *runProcess;
            *runProcess = readyQueue[i];
            readyQueue[i] = temp;
        }
    }
}
```

대기열에 프로세스가 진입했을 때 실행 중인 프로세스가 있는 경우, 대기열에 진입한 프로세스의 실행 시간이 적다면 실행하는 코드이다.

SRTF 스케줄링

입력

Process ID	Arrival Time	Burst Time
1	1	7
2	0	10
3	4	6
4	7	4
5	6	5

SRTF 스케줄링 실행

Process ID	Burst Time	WaitingTime
2	1	0
1	7	0
4	4	1
5	5	6
3	6	13
2	10	22

2	1		4	5		3	2	
0	1	8		12	17		23	32

먼저 도착한 process 2가 실행되는 중, 대기열로 진입한 process 1의 실행 시간이 더 짧으므로 process 1이 자원을 선점하여 실행되는 모습을 볼 수 있다.

결과

Process ID	Burst Time	WaitingTime
1	7	0
2	10	22
3	6	13
4	4	1
5	5	6

전체 실행 시간 : 32, 평균 대기 시간 : 8.4

SRTF 스케줄링 알고리즘의 결과이다. 평균 대기시간을 보면 FCFS에 비해 3.6초가 줄어든 것을 알 수 있다.

3.3 Preemptive Priority Scheduling Algorithm

다음은 Preemptive Priority 스케줄링 알고리즘을 구현하였다.

```
// 다음 실행될 프로세스 찾는 함수
int nextRun(Process* process, int size) {
    int index = 0;
    Process highPriority = *process;

    for (int i = 0; i < size; i++) {
        if (highPriority.processID == -1 && process[i].processID != -1) {
            highPriority = process[i];
            index = i;
            break;
        }
    }

    for (int i = 0; i < size; i++) {
        if (process[i].processID != -1) {
            // 다음 실행될 프로세스이기 때문에 대기열에 있는 프로세스 중 가장 우선순위가 높아야 함
            if (highPriority.priority > process[i].priority) {
                highPriority = process[i];
                index = i;
            }
            else if (highPriority.priority == process[i].priority && highPriority.arrivalTime > process[i].arrivalTime) {
                highPriority = process[i];
                index = i;
            }
            else if (highPriority.priority == process[i].priority && highPriority.arrivalTime == process[i].arrivalTime
                && highPriority.processID > process[i].processID ) {
                highPriority = process[i];
                index = i;
            }
        }
    }

    return index;
}
```

실행 중인 프로세스가 없는 경우, 대기열에서 대기 중인 프로세스 중 우선순위가 가장 높은(숫자가 낮은) 프로세스가 실행된다. 우선순위가 같은 경우에는 도착 시간이 빠른 프로세스가 다음에 실행되고, 우선순위와 도착 시간이 모두 같은 경우에는 프로세스 ID의 숫자가 적은 프로세스가 먼저 실행되도록 코드를 작성하였다.

```
// 실행 중인 프로세스가 없는 경우
if (runProcess->processID == -1) {
    int index = nextRun(readyQueue, queueIndex);
    *runProcess = readyQueue[index];
    Initialize(&readyQueue, index);
}
```

```

else { // 실행 중인 프로세스가 있는 경우 우선순위가 높은 프로세스가 실행 (숫자가 낮을수록 우선순위가 높음)
    for (int i = 0; i < newProcess; i++) {
        if (readyQueue[i].processID != -1 && runProcess->priority > readyQueue[i].priority) {
            for (int j = 0; j < numProcess; j++) {
                if (runProcess->processID == showProcess[j].processID) {
                    ShowResult(showProcess[j]);
                    gantt[ganttIndex] = showProcess[j];
                    ganttIndex++;
                    gantt = realloc(gantt, sizeof(ResultElement) * (ganttIndex+1));
                }
            }
            Process temp = *runProcess;
            *runProcess = readyQueue[i]; // 대기 중인 프로세스 실행
            readyQueue[i] = temp; // 대기 중인 프로세스가 있던 자리에 실행하던 프로세스
        }
    }
}
}

```

새로운 프로세스가 대기열에 도착했을 때 실행 중인 프로세스가 있어 우선순위를 비교해야 하는 경우 역시 우선순위가 높은 프로세스가 자원을 선점하도록 작성하였다.

Preemptive Priority 스케줄링

입력

Process ID	Arrival Time	Burst Time	Priority
1	1	7	2
2	0	10	7
3	4	6	5
4	7	4	2
5	6	5	1

전에 구현한 두 스케줄링 알고리즘과는 달리 우선순위가 관여되었다.

Preemptive Priority 스케줄링 실행

Process ID	Burst Time	WaitingTime
2	1	0
1	5	0
5	5	0
1	7	5
4	4	6
3	6	13
2	10	22

2	1	5	1	4	3	2	
0	1	6	11	13	17	23	32

먼저 도착한 process 2가 실행 중에 도착한 process 1의 우선순위가 높아, 자원을 할당받아 실행하게 되었지만 process 1이 실행 중에 도착한 process 5의 우선순위가 높으므로 process 5가 자원을 할당받아 실행하게 되었다는 것을 볼 수 있다.

결 과

Process ID	Burst Time	WaitingTime
1	7	5
2	10	22
3	6	13
4	4	6
5	5	0

전 체 실행 시 간 : 32, 평 균 대 기 시 간 : 18.4

Preemptive Priority 스케줄링 알고리즘의 결과를 보면, 스케줄링 알고리즘을 구현하기 전에 예상했듯이 문맥 교환 오버헤드로 인하여 FCFS 스케줄링 알고리즘보다 평균 대기 시간이 높다는 것을 알 수 있다.

아래는 프로세스 중 하나의 실행 시간을 다른 프로세스의 실행 시간보다 극단적으로 늘린 사례이다.

FCFS 스케줄링

입 력

Process ID	Arrival Time	Burst Time
1	1	7
2	0	100
3	4	6
4	7	4
5	6	5

결 과

Process ID	Burst Time	WaitingTime
1	7	99
2	100	0
3	6	103
4	4	111
5	5	107

전 체 실행 시 간 : 122, 평 균 대 기 시 간 : 84.0

Preemptive Priority 스케줄링

입 력

Process ID	Arrival Time	Burst Time	Priority
1	1	7	2
2	0	100	7
3	4	6	5
4	7	4	2
5	6	5	1

결 과

Process ID	Burst Time	WaitingTime
1	7	5
2	100	22
3	6	13
4	4	6
5	5	0

전 체 실행 시 간 : 122, 평 균 대 기 시 간 : 18.4

결과를 보면 FCFS 스케줄링의 경우에는 제일 먼저 대기열에 도착한 process 2의 실행 시간이 다른 프로세스의 실행 시간보다 극단적으로 길기 때문에 process 2가 종료될 때 까지 다른 프로세스가 대기하여야 했지만, Preemptive Priority 스케줄링의 경우에는 process 2의 우선순위가 가장 낮기 때문에 다른 프로세스에게 자원을 선점당하여 다른 프로세스가 모두 종료된 후는 우선순위에 따라 순차적으로 실행할 수 있었고, 평균 대기시간은 짧아진 것을 확인할 수 있다.

3.4 Round Robin Scheduling

마지막으로 구현할 스케줄링 알고리즘은 Round Robin 스케줄링이다.

```
// 1번 프로세스의 도착 시간으로 모든 프로세스의 도착 시간 통일
for (int i = 0; i < numProcess; i++)
    process[i].arrivalTime = process[0].arrivalTime;
```

RR 스케줄링은 프로세스의 도착 시간이 같다는 것을 가정으로 하기 때문에 process 1의 도착 시간을 모든 프로세스의 도착 시간으로 통일하였고, 타임 쿼텀 또한 통일할 수 있게 첫 번째 프로세스의 우선순위를 타임 쿼텀으로 사용하는 것을 아래 코드의 마지막 부분에 작성하였고, RR 스케줄링의 입력 값의 process 1의 우선순위(타임 쿼텀)는 3으로 설정하였다.

```
// 실행 중인 프로세스가 없는 경우
if (runProcess->processID == -1) {
    int index = nextRun(readyQueue, queueIndex);
    *runProcess = readyQueue[index];
    Initialize(&readyQueue, index);
}
else if (runProcess->processID != -1 && timeQuantum == 0) { // 실행 중인 프로세스가 있는 경우
    if (readyQueue[0].processID != -1) {
        for (int j = 0; j < numProcess; j++) {
            if (runProcess->processID == showProcess[j].processID) {
                ShowResult(showProcess[j]);
                gantt[ganttIndex] = showProcess[j];
                ganttIndex++;
                gantt = realloc(gantt, sizeof(ResultElement) * (ganttIndex+1));
            }
        }
        Process temp = *runProcess;
        *runProcess = readyQueue[0]; // 대기 중인 프로세스 실행
        Initialize(&readyQueue, 0); // 대기 중인 프로세스가 있던 자리에 실행하던 프로세스
        readyQueue[queueIndex] = temp;
    }
    timeQuantum = process[0].priority;
}

// 대기열 정렬
Shift(readyQueue, queueIndex);
// 대기열 정렬 함수
void Shift(Process process[], int size) {
    Process temp;
    for (int i = 0; i <= size; i++) {
        int j = 1;
        if (process[i].processID != -1 && process[i-j].processID == -1 && i - j >= 0) {
            temp = process[i-j];
            process[i-j] = process[i];
            process[i] = temp;
            j++;
        }
    }
}

// 다음 실행될 프로세스 찾는 함수
int nextRun(Process* process, int size) {
    int index = 0;
    Process highPriority = process[index];
    return index;
}

// 타임 쿼텀 소비 코드
timeQuantum--;
currentTime++;
```

RR 스케줄링 알고리즘은 First-in, First-out. 즉, 선입선출을 기반으로 한 알고리즘이기 때문에 실행 중인 프로세스가 없는 경우 항상 대기열의 가장 앞에 있는 프로세스에게 자원을 할당하기 위해 대기열을 정렬하는 함수를 생성하였고, 그로 인하여 nextRun() 함수의 코드는 상당히 간단해졌다. 타임 쿼텀의 작동을 구현하기 위해 프로세스를 실행하는 조건문에 타임 쿼텀 소비 코드를 추가하였다.

아래의 결과를 보면, 추가한 코드에 따라 도착 시간과 타임 쿼텀이 통일된 것을 볼 수 있다.

Round Robin 스케줄링

입력

Process ID	Arrival Time	Burst Time	Time Quantum
1	1	7	5
2	1	10	5
3	1	6	5
4	1	4	5
5	1	5	5

Round Robin 스케줄링 실행

Process ID	Burst Time	WaitingTime
1	5	0
2	5	5
3	5	10
4	4	15
5	5	19
1	7	19
2	10	21
3	6	26

1	2	3	4	5	1	2	3
0	5	10	15	19	24	26	31

실행 과정은 타임 쿼텀에 맞춰 적절하게 진행되었다는 것을 확인할 수 있다.

결과

Process ID	Burst Time	WaitingTime
1	7	19
2	10	21
3	6	26
4	4	15
5	5	19

전체 실행 시간 : 32, 평균 대기 시간 : 40.0

RR 스케줄링 역시 구현 전에 예상한 대로 타임 쿼텀이 작고, 프로세스의 도착이 모두 같기 때문에, 문맥 교환 오버헤드로 인해서 평균 대기시간이 상당히 다른 스케줄링 알고리즘에 비해서 상당히 큰 것을 알 수 있다.

Round Robin 스케줄링

입력

Process ID	Arrival Time	Burst Time	Time Quantum
1	1	7	12
2	1	10	12
3	1	6	12
4	1	4	12
5	1	5	12

Round Robin 스케줄링 실행

Process ID	Burst Time	WaitingTime
1	7	0
2	10	7
3	6	17
4	4	23
5	5	27

1	2	3	4	5
---	---	---	---	---

0 7 17 23 27 32

결과

Process ID	Burst Time	WaitingTime
1	7	0
2	10	7
3	6	17
4	4	23
5	5	27

전체 실행 시간 : 32, 평균 대기 시간 : 29.6

위의 결과는 타임 쿼텀을 12로 늘린 RR 스케줄링 알고리즘의 결과이다. 타임 쿼텀을 지나치게 크게 늘렸을 때 FCFS와 똑같이 작동하고, 평균 대기시간을 주목해 보면 오히려 하위 호환이라는 것을 알 수 있다.

하지만 본론에서 다루지 않은 사용자 요청에 대한 응답 시간과 같은 다른 기준으로 스케줄링 알고리즘을 평가한다면 분명히 이점이 있는 스케줄링 알고리즘으로 평가할 수 있다.

IV. 성능 평가

1. 실험 환경

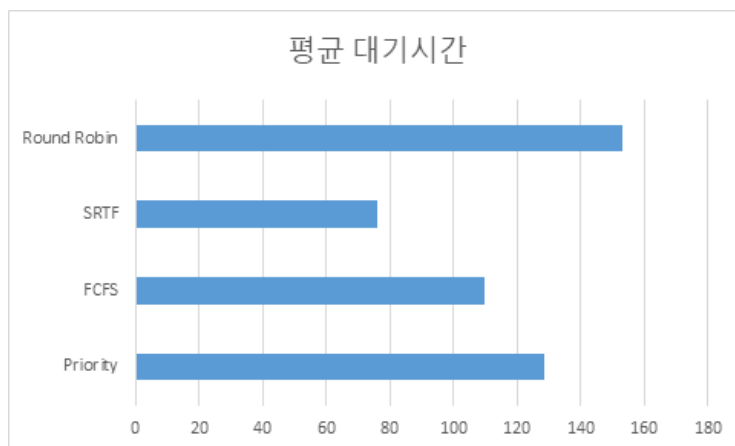
- 1) Visual Studio를 이용하여 Priority, FCFS, SJF, Round Robin를 c언어로 구현한다.
- 2) Priority, FCFS, SJF의 입력 데이터는 [process / PID / Arrive_Time / Burst_TIME / Priority] 으로, RR의 입력 데이터는 [process / PID / Arrive_Time / Burst_Time / Time_Quantum] 으로 통일한다.
- 3) 총 50개의 예시 데이터가 네 알고리즘에 동등하게 주어지며, 평균 / 최악의 경우로 나누어 평가한다. 이때 평균에 해당하는 예시 데이터는 랜덤으로 정해지고, 최악에 해당하는 데이터는 각기 다른 데이터를 제공한다.
- 4) Time Quantum의 경우 burst_time의 평균을 반올림하여 사용한다.
- 5) 유의미한 값이 존재하지 않을 때 '-'라 표기한다.
- 6) 최악의 경우에 해당하는 예시 데이터는 각각의 알고리즘에 해당하는 '최악의 상황'을 가정하여 다른 데이터를 제공한다.
- 7) 성능평가는 전체 실행시간 (스케줄링 오버헤드), 평균 대기시간, CPU 이용률, 메모리 사용량, 응답 시간을 기준으로 하여 비교한다.
- 8) FCFS의 최악 - 처리 시간이 긴 프로세스가 우선적으로 선점하여 '콘보이 효과'를 발생할 때를 가정한다.
- 9) SJF, Priority의 최악 - Starvation의 발생할 때를 가정한다.
- 10) RR의 최악 - Time Quantum이 매우 적을 때, 혹은 매우 클 때 발생을 가정한다.
- 11) 메모리 사용량, cpu 이용률의 경우 VISUAL STUDIO가 제공하는 '성능 프로파일러'를 이용하여 테스트하고 기입한다.

2. 실험결과 분석

	Priority	FCFS	SRTF	Round Robin
전체 실행시간	285	285	285	285
평균 대기시간	128.5	110	76.1	153
CPU 이용률	41%	27%	33%	45%
메모리 사용량	740KB	719KB	720KB	775KB
평균 응답시간	128.5	110	76.1	88

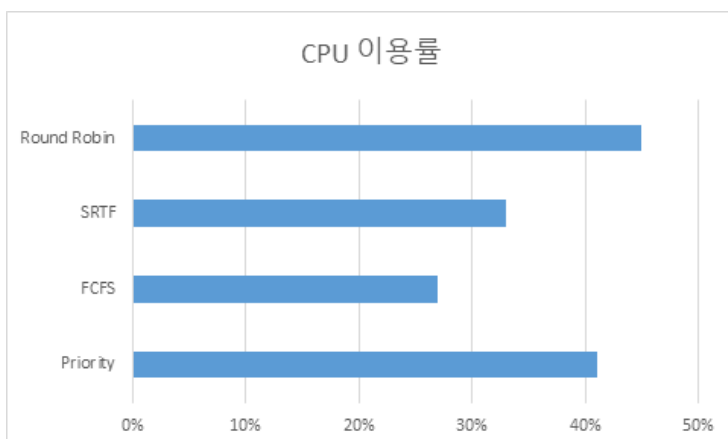
표 2 최악

- 평균 대기시간



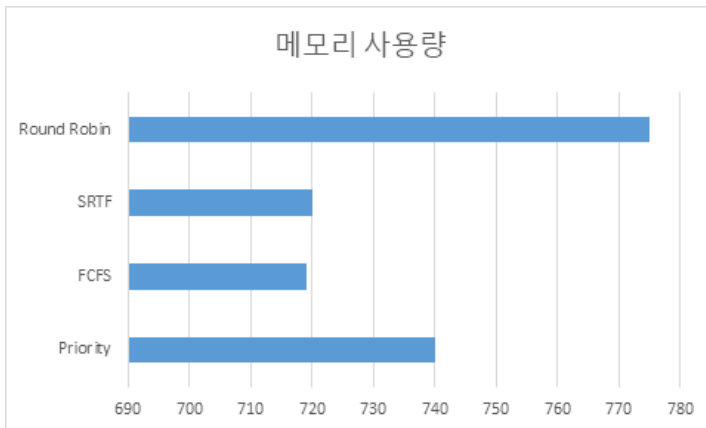
Priority와 SRTF의 경우 상대적으로 낮은 평균 대기시간을 가질 것으로 예상했으며, 실제로 Round Robin에 비해 짧은 대기시간을 나타내었다. 이는 Round Robin의 Context Switching(이하 문맥교환)으로 인한 것으로 예상된다. FCFS가 다른 세 알고리즘에 비해 유의미하게 짧은 대기시간을 가진 것을 확인 할 수 있다.

-CPU 이용률



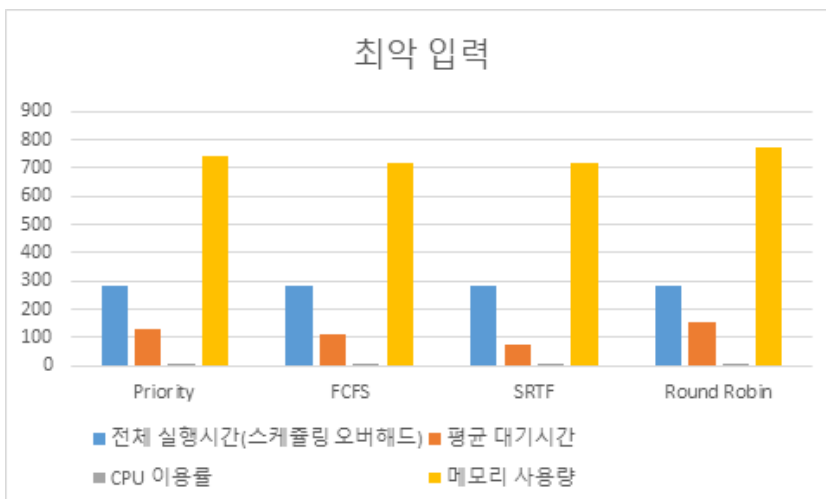
FCFS를 제외한 나머지 알고리즘은 평균에 비해 최악일 때 메모리 사용량과 cpu사용량이 유의미하게 늘어난 것을 확인할 수 있었다. Round Robin의 경우 다른 알고리즘에 비해 cpu의 사용량이 유의미하게 긴 것을 확인 할 수 있다.

- 메모리 사용량



메모리 사용량에는 큰 연관관계가 없을 것이라 예상했으나, Round Robin의 경우 타 알고리즘에 비해 더 높은 사용량을 보인다. 이는 Context Switching시 다음에 또 처리해야 하는 데이터를 저장해두기 때문에 그런 것이라 판단된다.

종합 -



결론적으로 최악의 데이터를 입력 할 시, SRTF가 가장 적은 평균 대기시간을 보였고 이는 짧은 프로그램을 우선으로 처리하는 특성 탓에 이러한 결과가 나온 것이라 예상된다. CPU 이용률과 메모리 사용량의 경우 FCFS가 가장 짧은 결론을 내었으며, 둘 모두 RR이 가장 큰 값을 도출해내었다.

이로인해 평균 대기시간을 최소화해야 하는 시스템의 경우 짧은 작업이 먼저 처리되는 SRTF의 알고리즘이 더욱 효율적임을 확인할 수 있다.

Priority와 SRTF의 경우 Starvation 현상이 발생했을 때 긴 대기시간을 갖게 되는 단점이 발견되었다. 또한, FCFS는 오버헤드가 적고 최악의 상황에서 전체 실행시간이 적은 것이 관찰되었다.

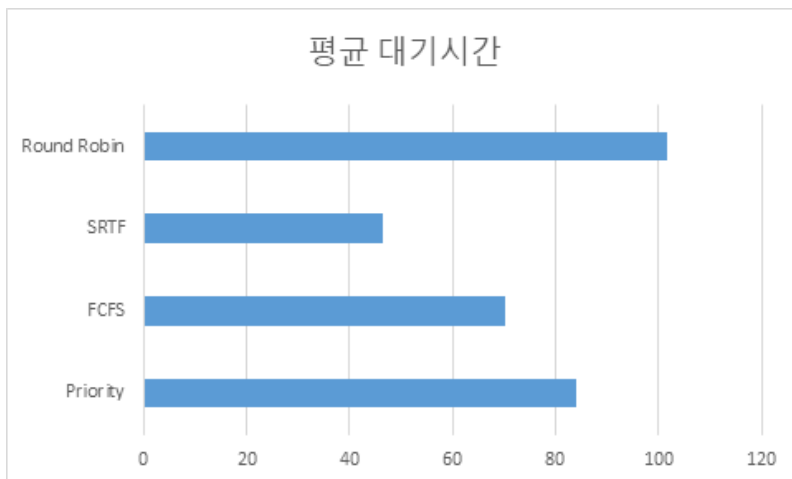
Round Robin의 경우 Time-Quantum에 따른 영향을 크게 받는 것을 확인할 수 있었다.

모든 알고리즘에서 평균에 비해 최악일 때 전체적인 모든 평가기준에 대해 성능이 떨어졌음을 확인 할 수 있다.

	Priority	FCFS	SJF	Round Robin
전체 실행시간 (스케줄링 오버헤드)	175	175	175	185
평균 대기시간	84	70.3	46.5	101.78
CPU 이용률	20%	27%	23%	26%
메모리 사용량	704KB	700KB	736KB	755KB
평균 응답 시간	84	70.3	55	46.7

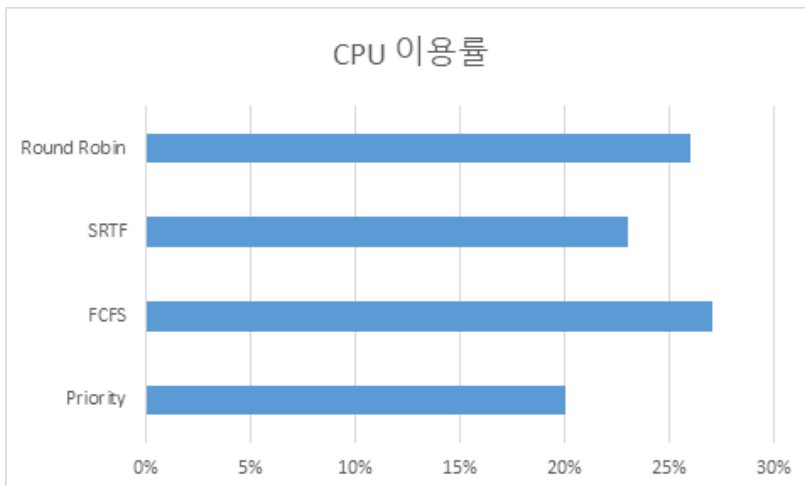
표 3 평균

– 평균 대기시간



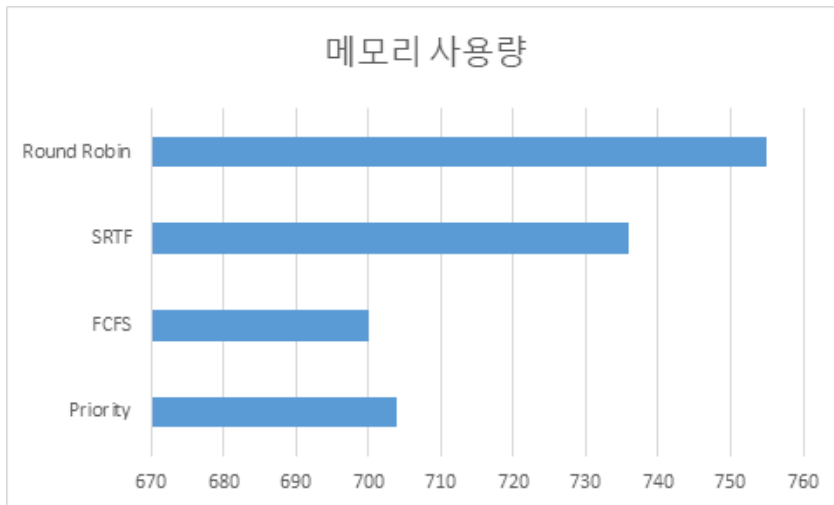
평균적으로 랜덤한 데이터를 입력하게 될 시 최악의 상황과 동일하게 SRTF가 가장 짧은 시간을 나타내었다.

– CPU 이용률



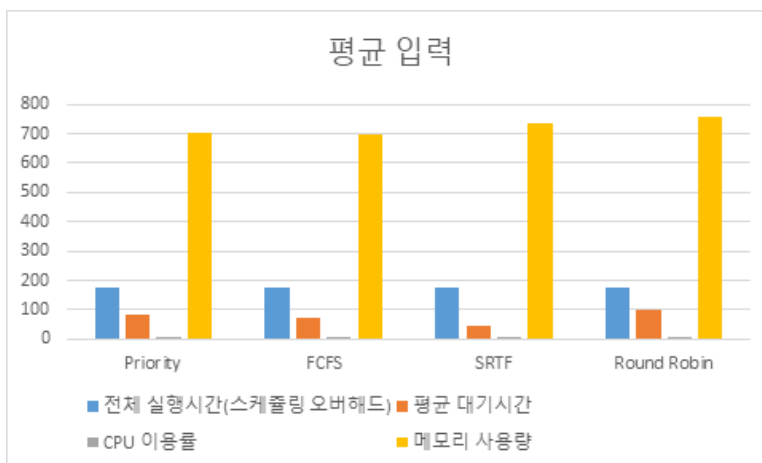
CPU의 이용률은 priority와 SRTF, Round Robin이 최악에 비해 그 비율이 유의미하게 줄어든 것을 확인할 수 있었으나, 그에 반해 FCFS는 유의미한 차이를 보이지 않았다.

- 메모리 사용량



메모리 사용량은 최악에 비해 전체적으로 크기가 작아진 것을 확인할 수 있다. 최악과 마찬가지로 FCFS가 가장 적은 값을 사용하였으며, RR이 가장 큰 값을 사용하였다. 또한, Priority가 다른 세 알고리즘에 비해 더 많은 증가율을 보였다.

- 종합



FCFS의 경우 최악의 상황, 즉 CPU를 짧게 쓰는 경우 오랜 시간을 기다려야하는 단점을 확인할 수 있다. 이는 앞 프로세스에 따라 기다리는 시간에 상당한 영향을 미치는 것을 뜻하게 된다.

SRTF의 경우 가장 적은 평균대기 시간이 가장 작다.

Round Robin의 경우 '공정한 사용'이 가능하다. 또한, CPU 사용시간이 짧은 프로세스와 긴 프로세스가 마구잡이로 섞여 있을 때(평균 입력)는 사용하기 좋은 스케줄러지만, CPU 사용시간이 비슷한 프로세스가 모여 있는 입력에서는 좋지 않은 성능을 확인할 수 있다. 평균 응답시간 (Average Turnaround Time)이 가장 적은 것을 확인할 수 있다.

V. 결론

본 프로젝트의 주요 목표는 다양한 CPU 스케줄링 알고리즘을 구현하고 성능을 분석하여, 각 알고리즘의 장단점을 평가하고 최적의 스케줄링 방안을 모색하는 것이다. 이를 통해 운영체제의 효율성과 성능을 향상시키고, 최신 기술 트렌드와 연구 방향을 이해하는 데 기여한다. 스케줄링 알고리즘 구현, 성능 분석 도구, 성능 측정 및 분석과 같은 핵심 기능을 제공하고 First-Come-First-Serve (FCFS), Shortest-Job-First Scheduling, Priority Scheduling, Round-Robin Scheduling과 같은 대표적인 스케줄링 알고리즘을 다룬다.

본 프로젝트에서는 다양한 CPU 스케줄링 알고리즘을 비교 분석하였다. Priority와 Shortest-Job-First 알고리즘은 낮은 평균 대기시간을 보였으나, Starvation 문제가 발생할 때 긴 대기시간을 가지는 단점이 있었다. Round-Robin(RR) 알고리즘은 컨텍스트 스위칭으로 인해 오버헤드가 크고, 메모리 사용량도 높았다. First-Come-First-Serve (FCFS) 알고리즘은 오버헤드가 적고 안정적이었지만, 평균 대기시간이 길어질 수 있었다. 모든 알고리즘에서 최악의 상황에서는 성능이 저하되었으며, 평균 대기시간을 최소화해야 하는 시스템에서는 Shortest-Job-First Scheduling 알고리즘이 가장 효율적임을 확인할 수 있었다.

모든 알고리즘에서 최악의 상황에서는 성능이 저하되었으며, 특히 메모리 사용량과 CPU 사용량이 높아짐을 확인했다. 따라서 시스템의 요구사항에 맞는 적절한 스케줄링 알고리즘을 선택하는 것이 중요함을 재확인하였다. 이러한 결론들은 운영체제 설계와 관련된 다양한 응용 분야에 기여할 것으로 기대된다.

향후 알고리즘 개선 방안으로는 여러 스케줄링 알고리즘의 장점을 결합한 하이브리드 스케줄링 알고리즘을 개발될 수 있다. 예를 들어, 시스템 부하가 낮을 때는 Shortest-Job-First Scheduling을, 부하가 높을 때는 Round-Robin Scheduling을 사용하는 등 상황에 따라 적절한 알고리즘을 선택하여 적용한다. 이를 통해 다양한 상황에서 일관된 성능을 제공하고, 각 알고리즘의 단점을 보완하여 전체 시스템의 성능을 향상시킬 수 있다.

참고 자료

- [1] 본 문서의 주요 참고 자료
- [2] 도서, 웹 사이트, 기술 문서 등